On-device speaker verification on an M4 Mac mini: which model to use, how to enroll, how to set a threshold

Goal: one enrolled speaker about 3 m from a cheap USB mic. Reject TV and podcast voices. Each 2 s clip must be scored in under ~20 ms with onnxruntime in Python. Prepared 2026-10-06. Every number has a source. **[M]** = I measured it in the sandbox (ARM Neoverse-V2, 2 vCPU, onnxruntime CPU EP; the M4 was not available). **[E]** = my estimate.

1. Comparison table

EER on VoxCeleb1-O cleaned. Unless a row says otherwise, scoring is cosine with no score normalization.

Model	EER Vox1-O-clean (source; norm)	Emb dim	Ready ONNX: exact URL (size)	Input to ONNX	Latency per 2 s (~200 frames)
WeSpeaker ResNet34	0.867 % raw; 0.787 % +AS-norm; 0.718 % +AS-norm+QMF. LM variant: 0.797 % raw, 0.723 % AS-norm, 0.659 % AS-norm+QMF [W1]. All WeSpeaker numbers subtract the vox2_dev mean embedding before cosine [W1].	256	HF: https://huggingface.co/Wespeaker/wespeaker-voxceleb-resnet34/resolve/main/voxceleb_resnet34.onnx?download=true (26.5 MB) and .../wespeaker-resnet34-LM/resolve/main/voxceleb_resnet34_LM.onnx?download=true [W2]. sherpa: https://github.com/k2-fsa/sherpa-onnx/releases/download/speaker-recongition-models/wespeaker_en_voxceleb_resnet34.onnx and `..._resnet34_LM.onnx` (26.5 MB each) [S1]	`feats` [B,T,80] -> `embs` [B,256] **[M]**	151 ms on 1 thread, 77 ms on 2 threads **[M]**. 4.55 GFLOPs [W1]. M4: ~30-50 ms **[E]**. **Over budget.**
WeSpeaker ResNet221 (LM)	0.569 % raw (non-LM), 0.479 % AS-norm. LM: 0.580 % raw, 0.505 % AS-norm [W1]	256	HF: https://huggingface.co/Wespeaker/wespeaker-voxceleb-resnet221-LM/resolve/main/voxceleb_resnet221_LM.onnx?download=true (95.0 MB). sherpa: .../speaker-recongition-models/wespeaker_en_voxceleb_resnet221_LM.onnx (95.0 MB) [S1]. No non-LM ONNX is published [W2].	`feats` [B,T,80] **[M]**	662 ms on 1 thread, 343 ms on 2 threads **[M]**. 21.29 GFLOPs [W1]. **Far over budget.**

Model	EER Vox1-O-clean (source; norm)	Emb dim	Ready ONNX: exact URL (size)	Input to ONNX	Latency per 2 s (~200 frames)
3D-Speaker ERes2Net-base (VoxCeleb)	0.97 % (base), 0.84 % (base-lm) [D2]. Same 0.84 % in the README benchmark [D1]. No AS-norm stated. WeSpeaker's re-implementation: 0.914 % raw / 0.803 % AS-norm [W1].	192 **[M]**	sherpa: https://github.com/k2-fsa/sherpa-onnx/releases/download/speaker-recongition-models/3dspeaker_speech_eres2net_sv_en_voxceleb_16k.onnx (26.5 MB; from modelscope iic/speech_eres2net_sv_en_voxceleb_16k) [S1]. Not stated whether it is base or base-lm.	`x` [N,T,80] -> `embedding` [*,192] **[M]**	119 ms on 1 thread, 64 ms on 2 threads **[M]**. Over budget.
3D-Speaker ERes2Net-large	0.57 % / 0.52 % (lm) [D2]	192 (inferred, not checked)	**No VoxCeleb ONNX found.** sherpa has only the Mandarin 3D-Speaker-trained file 3dspeaker_speech_eres2net_large_sv_zh-cn_3dspeaker_16k.onnx (116 MB) [S1].	`x` [N,T,80]	Not measured. 22.4 M params [D2], so expect worse than ResNet34.
3D-Speaker ERes2NetV2	0.68 % / 0.61 % (lm), VoxCeleb-trained [D3]	192 **[M]**	**No VoxCeleb ONNX.** Only the Mandarin 200k-speaker model: .../speaker-recongition-models/3dspeaker_speech_eres2netv2_sv_zh-cn_16k-common.onnx (71.4 MB) [S1]	`x` [N,T,80]	271 ms on 1 thread, 143 ms on 2 threads **[M]** (zh model). Over budget.
3D-Speaker CAM++ (VoxCeleb)	0.73 % (CAM++), 0.65 % (lm) [D4]. Model card says 0.73 % [D5]. No AS-norm stated.	512 **[M]**	sherpa: https://github.com/k2-fsa/sherpa-onnx/releases/download/speaker-recongition-models/3dspeaker_speech_campplus_sv_en_voxceleb_16k.onnx (29.6 MB) [S1]	`x` [N,T,80] -> `embedding` [*,512] **[M]**	47 ms on 1 thread, 28 ms on 2 threads **[M]**. Model card: CPU RTF 0.013 on 1 thread [D5].

Model	EER Vox1-O-clean (source; norm)	Emb dim	Ready ONNX: exact URL (size)	Input to ONNX	Latency per 2 s (~200 frames)
WeSpeaker CAM++	0.803 % raw, 0.718 % AS-norm. LM: 0.707 % raw, 0.659 % AS-norm [W1]	512	HF: https://huggingface.co/Wespeaker/wespeaker-voxceleb-campplus/resolve/main/voxceleb_CAM%2B%2B.onnx?download=true (29.3 MB) [W2]. sherpa: .../speaker-recongition-models/wespeaker_en_voxceleb_CAM%2B%2B.onnx and ..._CAM%2B%2B_LM.onnx (29.3 MB each) [S1]	`feats` [B,T,80] -> `embs` [B,512] **[M]**	47 ms on 1 thread, 28 ms on 2 threads **[M]**. 1.15 GFLOPs [W1]. M4 with 2-4 P-core threads: ~10-20 ms **[E]**
SpeechBrain ECAPA-TDNN (spkrec-ecapa-voxceleb)	0.80 % on Vox1-test cleaned, release 05-03-21. Norm not stated [B1].	192 [B2]	**No official ONNX.** PyTorch `embedding_model.ckpt` is 83.3 MB [B1]. You would export it yourself. WeSpeaker's own ECAPA512/1024 ONNX files are different models [W2].	SpeechBrain Fbank n_mels=80, sentence mean norm, no std norm [B2]. Raw waveform goes into its PyTorch pipeline.	Not measured
NeMo TitaNet-small	1.08 % on the "Voxceleb test trial file" (NGC v1.19.0). Norm not stated [N2].	192 **[M]**	sherpa: https://github.com/k2-fsa/sherpa-onnx/releases/download/speaker-recongition-models/nemo_en_titanet_small.onnx (40.3 MB) [S1]	`audio_signal` [B,80,T] + `length` int64 -> outputs `logits`, `embs` (192). Use output index 1. NeMo features: hann window, 25/10 ms, librosa mel, low_freq 0, per-feature mean/std norm, T padded to a multiple of 16 [S4] **[M]**	28 ms on 1 thread, **17 ms on 2 threads [M]**. Fastest, but the weakest EER.
NeMo TitaNet-large (for reference)	0.66 % VoxCeleb1 cleaned [N1]	192	.../speaker-recongition-models/nemo_en_titanet_large.onnx (101.4 MB) [S1]	same as TitaNet-small	120 ms on 1 thread, 62 ms on 2 threads **[M]**

Notes on the table: - The sherpa-onnx release tag really is spelled speaker-recongition-models. The asset list came from the GitHub API [S1]. - The sherpa-onnx WeSpeaker ONNX files are the official WeSpeaker exports plus sherpa metadata: `framework=wespeaker`, `normalize_samples=0`, `output_dim`, `sample_rate=16000`. The input and output names are unchanged (`feats`/`embs`) **[M]** [S2]. - The WeSpeaker "LM" (large-margin fine-tuned) models "could perform better on long audios, e.g. >3s" [W2]. With 2 s clips, the non-LM files may do as well or better. A/B test them on your own logs. - Latency figures are for the model only, measured as the

median of 15-20 runs. Kaldi fbank for 2 s took ~2.7 ms in Python **[M]**. I did not measure on M4. An M4 P-core should be faster than this sandbox vCPU, but that is an estimate. You can also try CoreMLExecutionProvider; sherpa-onnx lists `coreml` as a provider [S3]. Its speed-up is not measured.

2. Input format, and the trap between official WeSpeaker and sherpa-onnx

Pipeline	Sample scale	Fbank	CMN
Official WeSpeaker (wespeaker/bin/infer_onnx.py) [W3]	waveform x 32768 (int16 range)	torchaudio.compliance.kaldi.fbank: 80 bins, 25 ms / 10 ms, dither 0, **hamming**, use_energy False. Defaults: snip_edges True, high_freq = Nyquist.	**Yes**: mat-mean(mat,dim=0), no CVN
The WeSpeaker CLI does the same for ResNets [W4]		hamming. For CAM++ and ERes2Net checkpoints the CLI switches to **povey** and waveform norm [W4].	cmn=True
sherpa-onnx C++/Python for `framework=wespeaker` [S2][S5][S6]	x 32768 (`normalize_samples=0`)	knf defaults: **povey** window, snip_edges False, low 20 Hz, high -400 (7600 Hz)	**No.** feature_normalize_type is empty, so mean subtraction is skipped.
sherpa-onnx for `framework=3d-speaker` [S7]	[-1, 1] (`normalize_samples=1`)	80-bin knf fbank. The test script uses snip_edges True.	**Yes** (`global-mean`, meaning the per-utterance mean over frames)
sherpa-onnx NeMo (TitaNet) [S4]	[-1, 1]	hann, librosa mel, remove_dc False, low_freq 0	per-feature mean **and** std, then pad to a multiple of 16

What I measured [M], using sherpa's own test clips (Mandarin, close-talk): two 2 s enrollment clips for "fangjun", tested on another fangjun clip and two "leijun" clips. Script: /workspace/work/spk/scores.py.

Model / front-end	Same speaker	Different speakers
WeSpeaker ResNet34-LM, **no CMN** (sherpa path)	0.84	**0.81, 0.74** (scores collapse)
WeSpeaker ResNet34-LM, CMN + hamming (official)	0.54	0.31, 0.31
WeSpeaker ResNet221-LM, no CMN	0.91	0.56, **0.91**
WeSpeaker ResNet221-LM, CMN + hamming	0.66	0.29, 0.31
WeSpeaker CAM++ (non-LM), CMN + hamming	0.64	0.29, 0.21
WeSpeaker CAM++-LM, CMN + hamming	0.66	0.28, 0.28
3D-Speaker CAM++ VoxCeleb, CMN + povey	0.48	0.21, 0.21
TitaNet-small (NeMo front-end)	0.61	0.25, 0.35

Conclusion: **always apply CMN for WeSpeaker models.** Do not rely on sherpa-onnx's built-in extractor for WeSpeaker ResNets. CAM++ held up without CMN in my test, but use CMN anyway. Treat these scores as anecdotal: n = 3 trials, clean close-talk audio. Expect lower same-speaker scores at 3 m. I found **no published table of genuine vs impostor cosine distributions** for any of these models. The only published reference points are sherpa-onnx's example default thresholds: 0.6 in speaker-identification.py and 0.4 in speaker-identification-with-vad-dynamic.py [S3]. These are not calibrated values. Your current pyannote margin (same 0.2-0.38 vs other ~0.0) is worse than the ~0.3 gap between same- and different-speaker scores above, but you must re-measure in your room.

3. Recommendation

Use WeSpeaker CAM++ (VoxCeleb, non-LM), 512-dim, with the official WeSpeaker front-end (x32768, hamming, CMN). - File: https://github.com/k2-fsa/sherpa-onnx/releases/download/speaker-recongition-models/wespeaker_en_voxceleb_CAM%2B%2B.onnx (29.3 MB). The identical graph without metadata is at https://huggingface.co/Wespeaker/wespeaker-voxceleb-campplus/resolve/main/voxceleb_CAM%2B%2B.onnx?download=true. - Why: it is the only strong model (0.803 % raw / 0.718 % AS-norm [W1]) whose compute (1.15 GFLOPs, a quarter of ResNet34 [W1]) has a realistic chance of fitting 20 ms on an M4. Here it ran at 28 ms on 2 threads **[M]**. It also had the widest same-vs-different gap in my small test. - A/B test: wespeaker_en_voxceleb_CAM%2B%2B_LM.onnx. LM is better on long audio [W2]. - Fallback if you measure over 20 ms on the M4: nemo_en_titanet_small.onnx (17 ms on 2 threads here **[M]**, but 1.08 % EER [N2]). Or keep CAM++ and run it only on VAD-confirmed speech, so most audio never reaches the model. - ResNet221, ERes2Net-large and ERes2NetV2 are off the table for a 20 ms budget on CPU. - **Check first on the M4:** pythonverify_campplus.pymodel.onnxenr1.wavenr2.wavtest*.wav, then time `sess.run` over 200 frames with `intra_op_num_threads` set to 1, 2 and 4.

4. Enrollment

What the sources say, and what I added (marked): - **VAD and trimming**: run Silero VAD (MIT; under 1 ms per 30+ ms chunk on one CPU thread [V1]). Embed only the speech regions. sherpa's VAD example uses min_speech 1.0 s and min_silence 0.25 s [S3]. The WeSpeaker CLI can run Silero VAD before embedding [W4]. - **How many clips and how long** (practice, not from a source): 10-20 clips of 3-10 s of natural speech, about 1-3 minutes of net speech in total. Record them **at the real 3 m spot with the real USB mic**, in several sessions on different days or times of day. Do a set **with the TV off and with the TV on at normal volume**, and some at other positions in the room. Enrollment conditions should match test conditions. The 2 s test clips are short, so slicing enrollment into 2-3 s windows also gives matched-length embeddings for calibrating the threshold. - **Averaging vs keeping several**: L2-normalize each clip's embedding, average them, then re-normalize to get a centroid. Also **keep the individual embeddings**. Score = max(cos to centroid, mean of top-3 per-clip cosines), or keep one centroid per condition (TV-on and TV-off) and take the max. sherpa-onnx's own manager supports multi-clip enrollment through `add_multi`/`addMulti`, "for better accuracy" [S8]. Choosing between centroid and max is my practice; pick it on your logs. - **Mean subtraction**: WeSpeaker's published numbers subtract a domain mean embedding (vox2_dev) before cosine [W1]. The ONNX export does not include it: `mean_vec` defaults to zeros [W5]. To do the same, compute the mean raw embedding over a few hundred clips from *your room*: TV, podcasts, your own voice, other household members. Subtract it from every embedding before normalizing (build_enrollment(...,domain_mean=...)). That this helps in your setup is a reasonable expectation **[E]**; validate it against your logs.

5. Score normalization (AS-norm)

- Formula: s_norm = 1/2[(s-mu_e)/sigma_e + (s-mu_t)/sigma_t]. mu and sigma are the mean and std of the top-N cohort scores for the enrollment side and for the test side (WeSpeaker `score_norm.py`) [W5]. The WeSpeaker VoxCeleb recipe uses `top_n=300` with the vox2_dev cohort [W6]. It improves ResNet34 from 0.867 to 0.787 % and CAM++ from 0.803 to 0.718 % [W1].
- Matejka et al.: adaptive s-norm was best, about 30 % relative improvement. Best results came from selecting **200-500 top-scoring files** per side. The cohort should pool several languages and channels and "if possible, its subset should contain data from the target domain" [M1].
- **Your cohort**: about 1-3k 2 s embeddings. (a) TV and podcast audio recorded **through the room mic at the real position**, covering many shows, voices and ads. (b) A generic slice of other speakers (any VoxCeleb or LibriSpeech-like set) so the stats are stable. Never include the enrolled user. Start with top-k ~ 100-300; with a small cohort, keep k <~ 10-20 % of it. This sizing is my adaptation of [W6][M1].
- Cost: one 512xN matrix-vector product per test clip, about 0.1 ms for N = 2k **[E]**. The enrollment side's mu_e and sigma_e are fixed, so precompute them.
- Bonus: if a test clip's top cohort score is very high (it closely matches a known TV voice), that is direct evidence it is an impostor. You can log it as a feature **[E]**.

6. Picking and maintaining the threshold

1. **Log trials per condition.** Each record: raw cosine, AS-norm score, VAD speech duration, RMS level, timestamp, and a condition tag (user TV-off, user TV-on, TV alone, podcast alone, other household person). Get at least about 300 genuine and 1,000+ impostor 2 s clips. At FAR = 1 %, 1,000 impostors means only about 10 errors, so get 3k+ to set a reliable 0.1-1 % point.

2. **Compute the ROC/DET curve** for both raw and AS-norm scores (sklearn.metrics.roc_curve, or plot FRR against FAR on probit axes). The **EER point** is where FAR = FRR. Report it, but **don't use it as the operating point**: false accepts from the TV are the costly error.
3. **Choose the operating point by a target FAR**: the threshold theta is the (1 - FAR_target) quantile of the impostor scores, for example the 99th percentile for 1 % or the 99.9th for 0.1 %. Read off the FRR there. If FRR is too high, improve enrollment or the cohort rather than lowering theta.
4. **Add a margin.** Set theta slightly above that quantile, by about 0.5-1 sigma of the impostor score distribution or a fixed +0.02-0.05 in cosine. Also require **k-of-n consecutive accepts** (for example 2 of 3 overlapping 2 s windows) or a minimum of 1 s of VAD speech. This is my practice, not from a source.
5. **Check each condition separately.** The threshold must hold for the *worst* impostor condition, usually TV dialogue similar to the user's voice. Check FRR for user + TV-on separately too.
6. **Re-check over time.** Recompute the curves weekly or monthly from logged clips. Watch the genuine-score median for drift (mic, illness, room changes). Re-enroll or add clips when it drops. Re-tune if the household or TV content changes. Never auto-add accepted clips to enrollment without a high-confidence gate; it can poison the model.

7. Python snippet (tested in the sandbox on both CAM++ files)

The full file is /workspace/work/spk/verify_campplus.py. Core:

```
import numpy as np, onnxruntime as ort, kaldi_native_fbank as knf   # pip install onnxruntime kaldi-native-fbank

so = ort.SessionOptions(); so.intra_op_num_threads = 2; so.inter_op_num_threads = 1
sess = ort.InferenceSession("wespeaker_en_voxceleb_CAM++.onnx", so, providers=["CPUExecutionProvider"])
print(sess.get_inputs())   # expect: feats [B, T, 80] float  (3D-Speaker files: 'x' [N, T, 80])
print(sess.get_outputs())  # expect: embs  [B, 512]          (3D-Speaker files: 'embedding')
print(sess.get_modelmeta().custom_metadata_map)  # sherpa files: framework, normalize_samples, output_dim
IN, OUT = sess.get_inputs()[0].name, sess.get_outputs()[0].name

def fbank_cmn(wav16k_f32, scale=32768.0, window="hamming"):   # 3D-Speaker: scale=1.0, window="povey"
    o = knf.FbankOptions()
    o.frame_opts.samp_freq = 16000; o.frame_opts.frame_length_ms = 25; o.frame_opts.frame_shift_ms = 10
    o.frame_opts.dither = 0.0; o.frame_opts.window_type = window; o.frame_opts.snip_edges = True
    o.mel_opts.num_bins = 80; o.mel_opts.high_freq = 0.0
    fb = knf.OnlineFbank(o); fb.accept_waveform(16000, (wav16k_f32 * scale).tolist()); fb.input_finished()
    F = np.stack([fb.get_frame(i) for i in range(fb.num_frames_ready)]).astype(np.float32)
    return F - F.mean(0, keepdims=True)                       # CMN - required for WeSpeaker

# torchaudio alternative (exactly the official recipe):
#   import torch, torchaudio.compliance.kaldi as K
#   F = K.fbank(torch.from_numpy(wav)[None]*32768, num_mel_bins=80, frame_length=25, frame_shift=10,
#               dither=0.0, sample_frequency=16000, window_type='hamming', use_energy=False)
#   F = (F - F.mean(0)).numpy()

def embed(wav):  return sess.run([OUT], {IN: fbank_cmn(wav)[None]})[0][0]
l2n = lambda v: v / np.linalg.norm(v, axis=-1, keepdims=True)

# enrollment: VAD-trimmed clips from the 3 m position, TV on and off
enroll_E = l2n(np.stack([embed(c) for c in enroll_clips]))      # keep per-clip
centroid = l2n(enroll_E.mean(0))
```

```
def score(test_wav, cohort=None, top_k=200):
    t = l2n(embed(test_wav)); s = float(centroid @ t)
    if cohort is None: return s
    se = np.sort(cohort @ centroid)[::-1][:top_k]; st = np.sort(cohort @ t)[::-1][:top_k]
    return 0.5 * ((s - se.mean()) / se.std() + (s - st.mean()) / st.std())   # AS-norm
```

Sources

- [W1] WeSpeaker VoxCeleb v2 results: https://github.com/wenet-e2e/wespeaker/blob/master/examples/voxceleb/v2/README.md (read via raw.githubusercontent.com)
- [W2] WeSpeaker pretrained models: https://github.com/wenet-e2e/wespeaker/blob/master/docs/pretrained.md
- [W3] https://github.com/wenet-e2e/wespeaker/blob/master/wespeaker/bin/infer_onnx.py
- [W4] https://github.com/wenet-e2e/wespeaker/blob/master/wespeaker/cli/speaker.py
- [W5] https://github.com/wenet-e2e/wespeaker/blob/master/wespeaker/bin/score_norm.py and https://github.com/wenet-e2e/wespeaker/blob/master/wespeaker/bin/expo
- [W6] https://github.com/wenet-e2e/wespeaker/blob/master/examples/voxceleb/v2/run.sh (top_n=300, cohort vox2_dev)
- [D1] 3D-Speaker README: https://github.com/modelscope/3D-Speaker
- [D2] https://github.com/modelscope/3D-Speaker/blob/main/egs/voxceleb/sv-eres2net/README.md
- [D3] https://github.com/modelscope/3D-Speaker/blob/main/egs/voxceleb/sv-eres2netv2/README.md
- [D4] https://github.com/modelscope/3D-Speaker/blob/main/egs/voxceleb/sv-cam%2B%2B/README.md
- [D5] https://www.modelscope.cn/models/iic/speech_campplus_sv_en_voxceleb_16k/summary
- [S1] sherpa-onnx release: https://github.com/k2-fsa/sherpa-onnx/releases/tag/speaker-recongition-models (assets via api.github.com/repos/k2-fsa/sherpa-onnx/releases/tags/speaker-recongition-models). Docs: https://k2-fsa.github.io/sherpa/onnx/speaker-identification/index.html
- [S2] https://github.com/k2-fsa/sherpa-onnx/blob/master/scripts/wespeaker/add_meta_data.py and . . . /scripts/wespeaker/test.py (no CMN)
- [S3] https://github.com/k2-fsa/sherpa-onnx/blob/master/python-api-examples/speaker-identification.py (threshold 0.6) and . . . /speaker-identification-with-vad-dynamic.py (threshold 0.4, VAD settings, providers cpu/cuda/coreml)
- [S4] https://github.com/k2-fsa/sherpa-onnx/blob/master/scripts/nemo/speaker-verification/test-onnx.py and export-onnx.py
- [S5] https://github.com/k2-fsa/sherpa-onnx/blob/master/sherpa-onnx/csrc/speaker-embedding-extractor-general-impl.h
- [S6] https://github.com/k2-fsa/sherpa-onnx/blob/master/sherpa-onnx/csrc/features.h (povey, snip_edges false, high_freq -400)
- [S7] https://github.com/k2-fsa/sherpa-onnx/blob/master/scripts/3dspeaker/export-onnx.py and test-onnx.py
- [S8] https://k2-fsa.github.io/sherpa/onnx/javascript-api/examples/api_speaker_identification.html (addMulti, threshold trade-off)
- [B1] https://huggingface.co/speechbrain/spkrec-ecapa-voxceleb. [B2] hyperparams.yaml in the same repo.
- [N1] https://huggingface.co/nvidia/speakerverification_en_titanet_large. [N2] https://catalog.ngc.nvidia.com/orgs/nvidia/nemo/models/titanet_small/1.19.0/version-history
- [M1] Matejka et al., Interspeech 2017: https://www.fit.vut.cz/research/group/speech/public/publi/2017/matejka_interspeech2017_IS170803.pdf and https://www.fit.vut.cz/research/result/c144487/
- [V1] Silero VAD: https://github.com/snakers4/silero-vad